

To use `Linbcg`, below, you will need to supply routines that solve the auxiliary linear systems (2.7.42). If you have no idea what to use for the preconditioner $\tilde{\mathbf{A}}$, then use the diagonal part of $\mathbf{A}$, or even the identity matrix, in which case the burden of convergence will be entirely on the biconjugate gradient method itself.

`Linbcg`'s routine `solve`, below, is based on a program originally written by Anne Greenbaum. (See [11] for a different, less sophisticated, implementation.) There are a few wrinkles you should know about.

What constitutes “good” convergence is rather application-dependent. The routine `solve` therefore provides for four possibilities, selected by setting the flag `itol` on input. If `itol=1`, iteration stops when the quantity $|\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|/|\mathbf{b}|$ is less than the input quantity `tol`. If `itol=2`, the required criterion is

$$|\tilde{\mathbf{A}}^{-1} \cdot (\mathbf{A} \cdot \mathbf{x} - \mathbf{b})|/|\tilde{\mathbf{A}}^{-1} \cdot \mathbf{b}| < \text{tol} \quad (2.7.44)$$

If `itol=3`, the routine uses its own estimate of the error in $\mathbf{x}$ and requires its magnitude, divided by the magnitude of $\mathbf{x}$, to be less than `tol`. The setting `itol=4` is the same as `itol=3`, except that the largest (in absolute value) component of the error and largest component of $\mathbf{x}$ are used instead of the vector magnitude (that is, the L_∞ norm instead of the L_2 norm). You may need to experiment to find which of these convergence criteria is best for your problem.

On output, `err` is the tolerance actually achieved. If the returned count `iter` does not indicate that the maximum number of allowed iterations `itmax` was exceeded, then `err` should be less than `tol`. If you want to do further iterations, leave all returned quantities as they are and call the routine again. The routine loses its memory of the spanned conjugate gradient subspace between calls, however, so you should not force it to return more often than about every N iterations.

`linbcg.h`

```
struct Linbcg {
    Abstract base class for solving sparse linear equations by the preconditioned biconjugate gradient
    method. To use, declare a derived class in which the methods atimes and asolve are defined for
    your problem, along with any data that they need. Then call the solve method.
```

```
    virtual void asolve(VecDoub_I &b, VecDoub_O &x, const Int itrns) = 0;
    virtual void atimes(VecDoub_I &x, VecDoub_O &r, const Int itrns) = 0;
    void solve(VecDoub_I &b, VecDoub_IO &x, const Int itol, const Doub tol,
              const Int itmax, Int &iter, Doub &err);
    Doub snrm(VecDoub_I &xs, const Int itol);           Utility used by solve.
```

```
};
```

```
void Linbcg::solve(VecDoub_I &b, VecDoub_IO &x, const Int itol, const Doub tol,
                  const Int itmax, Int &iter, Doub &err)
```

Solves $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for $\mathbf{x}[0..n-1]$, given $\mathbf{b}[0..n-1]$, by the iterative biconjugate gradient method. On input $\mathbf{x}[0..n-1]$ should be set to an initial guess of the solution (or all zeros); `itol` is 1,2,3, or 4, specifying which convergence test is applied (see text); `itmax` is the maximum number of allowed iterations; and `tol` is the desired convergence tolerance. On output, $\mathbf{x}[0..n-1]$ is reset to the improved solution, `iter` is the number of iterations actually taken, and `err` is the estimated error. The matrix $\mathbf{A}$ is referenced only through the user-supplied routines `atimes`, which computes the product of either $\mathbf{A}$ or its transpose on a vector, and `asolve`, which solves $\tilde{\mathbf{A}} \cdot \mathbf{x} = \mathbf{b}$ or $\tilde{\mathbf{A}}^T \cdot \mathbf{x} = \mathbf{b}$ for some preconditioner matrix $\tilde{\mathbf{A}}$ (possibly the trivial diagonal part of $\mathbf{A}$). This routine can be called repeatedly, with `itmax` $\lesssim n$, to monitor how `err` decreases; or it can be called once with a sufficiently large value of `itmax` so that convergence to `tol` is achieved.

```
{
    Doub ak,akden,bk,bkden=1.0,bknum,bnrm,dxnrm,xnrm,zminrm,znrm;
    const Doub EPS=1.0e-14;
    Int j,n=b.size();
    VecDoub p(n),pp(n),r(n),rr(n),z(n),zz(n);
    iter=0;
    atimes(x,r,0);           Calculate initial residual.
    for (j=0;j<n;j++) {      Input to atimes is x[0..n-1], output is r[0..n-1];
        r[j]=b[j]-r[j];      the final 0 indicates that the matrix (not its
        rr[j]=r[j];           transpose) is to be used.
    }
}
```

```

//atimes(r,rr,0);
if (itol == 1) {
    bnmr=snrm(b,itol);
    asolve(r,z,0);
}
else if (itol == 2) {
    asolve(b,z,0);
    bnmr=snrm(z,itol);
    asolve(r,z,0);
}
else if (itol == 3 || itol == 4) {
    asolve(b,z,0);
    bnmr=snrm(z,itol);
    asolve(r,z,0);
    znmr=snrm(z,itol);
} else throw("illegal itol in linbcg");
while (iter < itmax) {
    ++iter;
    asolve(rr,zz,1);
    for (bknum=0.0,j=0;j<n;j++) bknum += z[j]*rr[j];
    Calculate coefficient bk and direction vectors p and pp.
    if (iter == 1) {
        for (j=0;j<n;j++) {
            p[j]=z[j];
            pp[j]=zz[j];
        }
    } else {
        bk=bknum/bkden;
        for (j=0;j<n;j++) {
            p[j]=bk*p[j]+z[j];
            pp[j]=bk*pp[j]+zz[j];
        }
    }
    bkden=bknum;
    atimes(p,z,0);
    for (akden=0.0,j=0;j<n;j++) akden += z[j]*pp[j];
    ak=bknum/akden;
    atimes(pp,zz,1);
    for (j=0;j<n;j++) {
        x[j] += ak*p[j];
        r[j] -= ak*z[j];
        rr[j] -= ak*zz[j];
    }
    asolve(r,z,0);
    if (itol == 1)
        err=snrm(r,itol)/bnrm;
    else if (itol == 2)
        err=snrm(z,itol)/bnrm;
    else if (itol == 3 || itol == 4) {
        zminrm=znmr;
        znmr=snrm(z,itol);
        if (abs(zminrm-znmr) > EPS*znmr) {
            dxnmr=abs(ak)*snrm(p,itol);
            err=znmr/abs(zminrm-znmr)*dxnmr;
        } else {
            err=znmr/bnmr;
        }
    }
    xnmr=snrm(x,itol);
    if (err <= 0.5*xnmr) err /= xnmr;
    else {
        err=znmr/bnmr;
    }
}

```

Uncomment this line to get the “minimum residual” variant of the algorithm.

Input to asolve is $r[0..n-1]$, output is $z[0..n-1]$; the final 0 indicates that the matrix $\tilde{\mathbf{A}}$ (not its transpose) is to be used.

Main loop.

Final 1 indicates use of transpose matrix $\tilde{\mathbf{A}}^T$.

Calculate coefficient a_k , new iterate x , and new residuals r and rr .

Solve $\tilde{\mathbf{A}} \cdot \mathbf{z} = \mathbf{r}$ and check stopping criterion.

Error may not be accurate, so loop again.

Error may not be accurate, so loop again.

```

    }
    if (err <= tol) break;
}
}

```

The solve routine uses this short utility for computing vector norms:

```

linbcg.h  Doub Linbcg::snrm(VecDoub_I &sx, const Int itol)
Compute one of two norms for a vector sx[0..n-1], as signaled by itol. Used by solve.
{
    Int i,isamax,n=sx.size();
    Doub ans;
    if (itol <= 3) {
        ans = 0.0;
        for (i=0;i<n;i++) ans += SQR(sx[i]);      Vector magnitude norm.
        return sqrt(ans);
    } else {
        isamax=0;
        for (i=0;i<n;i++) {                      Largest component norm.
            if (abs(sx[i]) > abs(sx[isamax])) isamax=i;
        }
        return abs(sx[isamax]);
    }
}

```

Here is an example of a derived class that solves $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for a matrix $\mathbf{A}$ in NRsparseMat's compressed column sparse format. A naive diagonal preconditioner is used.

```

asolve.h  struct NRsparseLinbcg : Linbcg {
            NRsparseMat &mat;
            Int n;
            NRsparseLinbcg(NRsparseMat &matrix) : mat(matrix), n(mat.nrows) {}
            The constructor just binds a reference to your sparse matrix, making it available to asolve
            and atimes. To solve for a right-hand side, you call this object's solve method, as defined
            in the base class.
            void atimes(VecDoub_I &x, VecDoub_O &r, const Int itrns) {
                if (itrns) r=mat.atx(x);
                else r=mat.ax(x);
            }
            void asolve(VecDoub_I &b, VecDoub_O &x, const Int itrns) {
                Int i,j;
                Doub diag;
                for (i=0;i<n;i++) {
                    diag=0.0;
                    for (j=mat.col_ptr[i];j<mat.col_ptr[i+1];j++)
                        if (mat.row_ind[j] == i) {
                            diag=mat.val[j];
                            break;
                        }
                    x[i]=(diag != 0.0 ? b[i]/diag : b[i]);
                    The matrix  $\tilde{\mathbf{A}}$  is the diagonal part of  $\mathbf{A}$ . Since the transpose matrix has the same
                    diagonal, the flag itrns is not used in this example.
                }
            }
        };

```

For another example of using a class derived from Linbcg to solve a sparse matrix problem, see §3.8.

CITED REFERENCES AND FURTHER READING:

Tewarson, R.P. 1973, *Sparse Matrices* (New York: Academic Press).[1]

- Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter I.3 (by J.K. Reid).[2]
- George, A., and Liu, J.W.H. 1981, *Computer Solution of Large Sparse Positive Definite Systems* (Englewood Cliffs, NJ: Prentice-Hall).[3]
- NAG Fortran Library (Oxford, UK: Numerical Algorithms Group), see 2007+, <http://www.nag.co.uk>.[4]
- IMSL Math/Library Users Manual (Houston: IMSL Inc.), see 2007+, <http://www.vni.com/products/ims1>.[5]
- Eisenstat, S.C., Gursky, M.C., Schultz, M.H., and Sherman, A.H. 1977, *Yale Sparse Matrix Package*, Technical Reports 112 and 114 (Yale University Department of Computer Science).[6]
- Bai, Z., Demmel, J., Dongarra, J. Ruhe, A., and van der Vorst, H. (eds.) 2000, *Templates for the Solution of Algebraic Eigenvalue Problems: A Practical Guide* Ch. 10 (Philadelphia: S.I.A.M.). Online at URL in <http://www.cs.ucdavis.edu/~bai/ET/contents.html>.[7]
- SPARSKIT, 2007+, at <http://www-users.cs.umn.edu/~saad/software/SPARSKIT/sparskit.html>.[8]
- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapters 4 and 10, particularly §10.2–§10.3.[9]
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 8.[10]
- Baker, L. 1991, *More C Tools for Scientists and Engineers* (New York: McGraw-Hill).[11]
- Fletcher, R. 1976, in *Numerical Analysis Dundee 1975*, Lecture Notes in Mathematics, vol. 506, A. Dold and B. Eckmann, eds. (Berlin: Springer), pp. 73–89.[12]
- PCGPAK User's Guide (New Haven: Scientific Computing Associates, Inc.).[13]
- Saad, Y., and Schultz, M. 1986, *SIAM Journal on Scientific and Statistical Computing*, vol. 7, pp. 856–869.[14]
- Ueberhuber, C.W. 1997, *Numerical Computation: Methods, Software, and Analysis*, 2 vols. (Berlin: Springer), Chapter 13.
- Bunch, J.R., and Rose, D.J. (eds.) 1976, *Sparse Matrix Computations* (New York: Academic Press).
- Duff, I.S., and Stewart, G.W. (eds.) 1979, *Sparse Matrix Proceedings 1978* (Philadelphia: S.I.A.M.).

2.8 Vandermonde Matrices and Toeplitz Matrices

In §2.4 the case of a tridiagonal matrix was treated specially, because that particular type of linear system admits a solution in only of order N operations, rather than of order N^3 for the general linear problem. When such particular types exist, it is important to know about them. Your computational savings, should you ever happen to be working on a problem that involves the right kind of particular type, can be enormous.

This section treats two special types of matrices that can be solved in of order N^2 operations, not as good as tridiagonal, but a lot better than the general case. (Other than the operations count, these two types having nothing in common.) Matrices of the first type, termed *Vandermonde matrices*, occur in some problems having to do with the fitting of polynomials, the reconstruction of distributions from their moments, and also other contexts. In this book, for example, a Vandermonde problem crops up in §3.5. Matrices of the second type, termed *Toeplitz matrices*, tend